

Persist Data Offline

Persist Data with 3 Ways

Eng | Ruqaih Hussein Salman

2025-2026

Ways to Store Data Offline in Flutter

There are many cases where you need to store data locally on users' devices instead of using remote cloud servers or APIs, for example, you may need to persist data across app launches, or download a language dictionary from the internet and save it for later offline use. This article will walk you through 3 solutions to do that.



Flutter Persistence Options

The persistence options you have on Flutter include:

- Local database using SQLite
- Key-Value Store (SharedPreferences).
- File storage (Read and write files).

You'll investigate three of these options in this lecture..

Persist data with SQLite

If you are writing an app that needs to persist and query large amounts of data on the local device, consider using a database instead of a local file or key-value store. In general, databases provide faster inserts, updates, and queries compared to other local persistence solutions.

SQL Database Storage Using Sqlite In Flutter



Flutter apps can make use of the SQLite databases via the [sqflite](#) plugin available on pub.dev. This recipe demonstrates the basics of using [sqflite](#) to insert, read, update, and remove data about various Dogs.

This recipe uses the following steps:

1. Add the dependencies.
2. Define the Dog data model.
3. Open the database.
4. Create the dogs table.
5. Insert a Dog into the database.
6. Retrieve the list of dogs.
7. Update a Dog in the database.
8. Delete a Dog from the database.

1. Add the dependencies

To work with SQLite databases, import the [sqflite](#) and [path](#) packages.

- The [sqflite](#) package provides classes and functions to interact with a SQLite database.
- The [path](#) package provides functions to define the location for storing the database on disk.

```
dependencies:  
  flutter:  
    sdk: flutter  
  sqflite:  
  path:
```

Make sure to import the packages in the file you'll be working in.

```
import 'dart:async';
```

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
```

2. Define the Dog data model

Before creating the table to store information on Dogs, take a few moments to define the data that needs to be stored. For this example, define a Dog class that contains three pieces of data: A unique `id`, the `name`, and the `age` of each dog.

```
class Dog {
  final int id;
  final String name;
  final int age;

  Dog({
    required this.id,
    required this.name,
    required this.age,
  });
}
```

3. Open the database

Before reading and writing data to the database, open a connection to the database. This involves two steps:

- Define the path to the database file using `getDatabasesPath()` from the `sqflite` package, combined with the `join` function from the `path` package.
- Open the database with the `openDatabase()` function from `sqflite`.

Note: In order to use the keyword `await`, the code must be placed inside an `async` function. You should place all the following table functions inside `void main() async {}`.

```
// Avoid errors caused by flutter upgrade.
// Importing 'package:flutter/widgets.dart' is required.
WidgetsFlutterBinding.ensureInitialized();
// Open the database and store the reference.
final database = openDatabase(
  // Set the path to the database. Note: Using the `join` function from the
  // `path` package is best practice to ensure the path is correctly
  // constructed for each platform.
  join(await getDatabasesPath(), 'doggie_database.db'),
);
```

4. Create the dogs table

Next, create a table to store information about various Dogs. For this example, create a table called `dogs` that defines the data that can be stored. Each `Dog` contains an `id`, `name`, and `age`. Therefore, these are represented as three columns in the `dogs` table.

- The `id` is a Dart `int`, and is stored as an `INTEGER` SQLite Datatype. It is also good practice to use an `id` as the primary key for the table to improve query and update times.
- The `name` is a Dart `String`, and is stored as a `TEXT` SQLite Datatype.

- The `age` is also a Dart `int`, and is stored as an `INTEGER` Datatype.

```
final database = openDatabase(  
  // Set the path to the database. Note: Using the `join` function from the  
  // `path` package is best practice to ensure the path is correctly  
  // constructed for each platform.  
  join(await getDatabasesPath(), 'doggie_database.db'),  
  // When the database is first created, create a table to store dogs.  
  onCreate: (db, version) {  
    // Run the CREATE TABLE statement on the database.  
    return db.execute(  
      'CREATE TABLE dogs(id INTEGER PRIMARY KEY, name TEXT, age INTEGER)',  
    );  
  },  
  // Set the version. This executes the onCreate function and provides a  
  // path to perform database upgrades and downgrades.  
  version: 1,  
);
```

5. Insert a Dog into the database

Now that you have a database with a table suitable for storing information about various dogs, it's time to read and write data.

First, insert a Dog into the dogs table. This involves two steps:

- Convert the `Dog` into a `Map`
- Use the `insert()` method to store the `Map` in the `dogs` table.

```
class Dog {  
  final int id;  
  final String name;  
  final int age;  
  
  Dog({  
    required this.id,  
    required this.name,  
    required this.age,  
  });  
  
  // Convert a Dog into a Map. The keys must correspond to the names of the columns in  
  // the database.  
  Map<String, dynamic> toMap() {  
    return {  
      'id': id,  
      'name': name,  
      'age': age,  
    };  
  }  
  
  // Implement toString to make it easier to see information about  
  // each dog when using the print statement.  
  @override  
  String toString() {  
    return 'Dog{id: $id, name: $name, age: $age}';  
  }  
}
```

```
// Define a function that inserts dogs into the database
Future<void> insertDog(Dog dog) async {
  // Get a reference to the database.
  final db = await database;

  // Insert the Dog into the correct table. You might also specify the
  // `conflictAlgorithm` to use in case the same dog is inserted twice.
  //
  // In this case, replace any previous data.
  await db.insert(
    'dogs',
    dog.toMap(),
    conflictAlgorithm: ConflictAlgorithm.replace,
  );
}

// Create a Dog and add it to the dogs table
var fido = Dog(
  id: 0,
  name: 'Fido',
  age: 35,
);

await insertDog(fido);
```

6. Retrieve the list of Dogs

Now that a `Dog` is stored in the database, query the database for a specific dog or a list of all dogs. This involves two steps:

- Run a `query` against the `dogs` table. This returns a `List<Map>`.
- Convert the `List<Map>` into a `List<Dog>`.

```
// A method that retrieves all the dogs from the dogs table.
Future<List<Dog>> dogs() async {
  // Get a reference to the database.
  final db = await database;

  // Query the table for all The Dogs.
  final List<Map<String, dynamic>> maps = await db.query('dogs');

  // Convert the List<Map<String, dynamic> into a List<Dog>.
  return List.generate(maps.length, (i) {
    return Dog(
      id: maps[i]['id'],
      name: maps[i]['name'],
      age: maps[i]['age'],
    );
  });
}

// Now, use the method above to retrieve all the dogs.
print(await dogs()); // Prints a list that include Fido.
```

7. Update a Dog in the database

After inserting information into the database, you might want to update that information at a later time. You can do this by using the `update()` method from the `sqlite` library.

This involves two steps:

- Convert the Dog into a Map.
- Use a `where` clause to ensure you update the correct Dog.

```
Future<void> updateDog(Dog dog) async {
  // Get a reference to the database.
  final db = await database;

  // Update the given Dog.
  await db.update(
    'dogs',
    dog.toMap(),
    // Ensure that the Dog has a matching id.
    where: 'id = ?',
    // Pass the Dog's id as a whereArg to prevent SQL injection.
    whereArgs: [dog.id],
  );
}

// Update Fido's age and save it to the database.
fido = Dog(
  id: fido.id,
  name: fido.name,
  age: fido.age + 7,
);
await updateDog(fido);

// Print the updated results.
print(await dogs()); // Prints Fido with age 42.
```

Warning: Always use `whereArgs` to pass arguments to a `where` statement. This helps safeguard against SQL injection attacks. Do not use string interpolation, such as `where: "id = ${dog.id}"`!

8. Delete a Dog from the database

In addition to inserting and updating information about Dogs, you can also remove dogs from the database. To delete data, use the `delete()` method from the `sqflite` library. In this section, create a function that takes an id and deletes the dog with a matching id from the database. To make this work, you must provide a `where` clause to limit the records being deleted.

```
Future<void> deleteDog(int id) async {
  // Get a reference to the database.
  final db = await database;

  // Remove the Dog from the database.
  await db.delete(
    'dogs',
    // Use a `where` clause to delete a specific dog.
    where: 'id = ?',
    // Pass the Dog's id as a whereArg to prevent SQL injection.
    whereArgs: [id],
  );
}
```

➤ Example: Complete Code of sqflite with dart

To run the example:

- Create a new Flutter project.
- Add the [sqflite](#) and [path](#) packages to your [pubspec.yaml](#).
- Paste the following code into a new file called [lib/db_test.dart](#).
- Run the code with [flutter run lib/db_test.dart](#).

```
import 'dart:async';
import 'dart:async';
import 'package:flutter/widgets.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

void main() async {
  // Avoid errors caused by flutter upgrade.
  // Importing 'package:flutter/widgets.dart' is required.
  WidgetsFlutterBinding.ensureInitialized();
  // Open the database and store the reference.
  final database = openDatabase(
    // Set the path to the database. Note: Using the `join` function from the
    // `path` package is best practice to ensure the path is correctly
    // constructed for each platform.
    join(await getDatabasesPath(), 'doggie_database.db'),
    // When the database is first created, create a table to store dogs.
    onCreate: (db, version) {
      // Run the CREATE TABLE statement on the database.
      return db.execute(
        'CREATE TABLE dogs(id INTEGER PRIMARY KEY, name TEXT, age INTEGER)',
      );
    },
    // Set the version. This executes the onCreate function and provides a
    // path to perform database upgrades and downgrades.
    version: 1,
  );

  // Define a function that inserts dogs into the database
  Future<void> insertDog(Dog dog) async {
    // Get a reference to the database.
    final db = await database;

    // Insert the Dog into the correct table. You might also specify the
    // `conflictAlgorithm` to use in case the same dog is inserted twice.
    //
    // In this case, replace any previous data.
    await db.insert(
      'dogs',
      dog.toMap(),
      conflictAlgorithm: ConflictAlgorithm.replace,
    );
  }

  // A method that retrieves all the dogs from the dogs table.
  Future<List<Dog>> dogs() async {
    // Get a reference to the database.
    final db = await database;

    // Query the table for all The Dogs.
    final List<Map<String, dynamic>> maps = await db.query('dogs');
```

```

// Convert the List<Map<String, dynamic> into a List<Dog>.
return List.generate(maps.length, (i) {
  return Dog(
    id: maps[i]['id'],
    name: maps[i]['name'],
    age: maps[i]['age'],
  );
});
}

Future<void> updateDog(Dog dog) async {
  // Get a reference to the database.
  final db = await database;

  // Update the given Dog.
  await db.update(
    'dogs',
    dog.toMap(),
    // Ensure that the Dog has a matching id.
    where: 'id = ?',
    // Pass the Dog's id as a whereArg to prevent SQL injection.
    whereArgs: [dog.id],
  );
}

Future<void> deleteDog(int id) async {
  // Get a reference to the database.
  final db = await database;

  // Remove the Dog from the database.
  await db.delete(
    'dogs',
    // Use a `where` clause to delete a specific dog.
    where: 'id = ?',
    // Pass the Dog's id as a whereArg to prevent SQL injection.
    whereArgs: [id],
  );
}

// Create a Dog and add it to the dogs table
var fido = Dog(
  id: 0,
  name: 'Fido',
  age: 35,
);

await insertDog(fido);

// Now, use the method above to retrieve all the dogs.
print(await dogs()); // Prints a list that include Fido.

// Update Fido's age and save it to the database.
fido = Dog(
  id: fido.id,
  name: fido.name,
  age: fido.age + 7,
);
await updateDog(fido);

// Print the updated results.

```

```

print(await dogs()); // Prints Fido with age 42.

// Delete Fido from the database.
await deleteDog(fido.id);

// Print the list of dogs (empty).
print(await dogs());
}

class Dog {
  final int id;
  final String name;
  final int age;

  Dog({
    this.id,
    this.name,
    this.age,
  });

  // Convert a Dog into a Map. The keys must correspond to the names of the
  // columns in the database.
  Map<String, dynamic> toMap() {
    return {
      'id': id,
      'name': name,
      'age': age,
    };
  }

  // Implement toString to make it easier to see information about
  // each dog when using the print statement.
  @override
  String toString() {
    return 'Dog{id: $id, name: $name, age: $age}';
  }
}

```

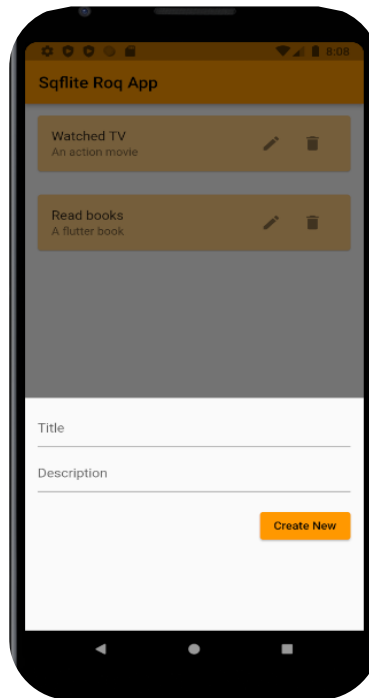
Output:

```

[Dog{id: 0, name: Fido, age: 35}]
[Dog{id: 0, name: Fido, age: 42}]

```

➤ Example: SQLite with Flutter widget



SQLite is a fast relational database that can be used to store data offline for mobile applications. CRUD means create, read, update, and delete, the four essential operations of persistent storage. In this section, we are going to build a small Flutter app that uses SQLite to persist data.

App preview:

The app we are going to make is an offline diary that lets users record the activities they did during the day. For simplicity's sake, we will call each of these activities a "journal" or an "item". In the scope of this tutorial, "activity", "item" and "journal" are interchangeable. The app has a floating button that can be used to show a bottom sheet. That bottom sheet contains 2 text fields corresponding to "title" and "description". These text fields are used to create a new "item" or update an existing "item". The saved "items" are fetched from the SQLite database and displayed with a list view. There are an update button and a delete button associated with each "item".

Database Structure

Advertisements

We are going to create an SQLite database called `activity_ruqaih_salman.db`. It has only a single table named `items`. Below is the structure of the table:

Column	Type	Description
id	INTEGER	The id of an activity
title	TEXT	The name of an activity
description	TEXT	The detail of an activity

createdAt	TIMESTAMP	The time that the item was created. It will be automatically added by SQLite
-----------	------------------	---

➤ The Complete Code of sqflite

1. Create a new Flutter project. In the lib folder, add a new file named `Sql_Helper_roq.dart`.

The project structure:

```
.
├── main.dart
└── Sql_Helper_roq.dart
```

Install the [sqflite](#) plugin (note that the name has an "f"):

```
flutter pub add sqflite
```

2. Full code in `Sql_Helper_roq.dart`:

```
import 'package:flutter/foundation.dart';
import 'package:sqflite/sqflite.dart' as sql;

class SQLHelper {
  static Future<void> createTables(sql.Database database) async {
    await database.execute("""CREATE TABLE items(
      id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
      title TEXT,
      description TEXT,
      createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    )""");
  }
  // id: the id of a item
  // title : name
  // description: description of your activity
  // created_at: the time that the item was created.
  // It will be automatically handled by SQLite

  static Future<sql.Database> db() async {
    return sql.openDatabase(
      'activity_ruqaih_salman.db',
      version: 1,
      onCreate: (sql.Database database, int version) async {
        await createTables(database);
      },
    );
  }

  // Create new item (journal)
  static Future<int> createItem(String title, String descrption) async {
    final db = await SQLHelper.db();

    final Map<String, String> data = {'title': title, 'description': descrption};
    final int id = await db.insert('items', data,
      conflictAlgorithm: sql.ConflictAlgorithm.replace);
  }
}
```

```

    return id;
  }

  // Read all items (journals)
  static Future<List<Map<String, dynamic>>> getItems() async {
    final db = await SQLHelper.db();
    return db.query('items', orderBy: "id");
  }

  // Read a single item by id
  // The app doesn't use this method but I put here in case you want to see it
  static Future<List<Map<String, dynamic>>> getItem(int id) async {
    final db = await SQLHelper.db();
    return db.query('items', where: "id = ?", whereArgs: [id], limit: 1);
  }

  // Update an item by id
  static Future<int> updateItem(
    int id, String title, String description) async {
    final db = await SQLHelper.db();

    final Map<String, String> data = {
      'title': title,
      'description': description,
      'createdAt': DateTime.now().toString()
    };

    final result =
      await db.update('items', data, where: "id = ?", whereArgs: [id]);
    return result;
  }

  // Delete
  static Future<void> deleteItem(int id) async {
    final db = await SQLHelper.db();
    try {
      await db.delete("items", where: "id = ?", whereArgs: [id]);
    } catch (err) {
      debugPrint("Something went wrong when deleting an item: $err");
    }
  }
}

```

3. Full code in main.dart (with explanations):

```

import 'package:flutter/material.dart';
import 'Sql_Helper._roq.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'Ruqaih_Data_Storage_offline.com',
    theme: ThemeData(
      primarySwatch: Colors.orange,
    ),
    home: MyUi_Page_Sqflite_Roq(),
  ));
}

class MyUi_Page_Sqflite_Roq extends StatefulWidget {
  @override

```

```

    _HomePageState createState() => _HomePageState();
}

class _HomePageState extends State<MyUi_Page_Sqflite_Roq> {
  // All journals
  List<Map<String, dynamic>> _journals = [];

  bool _isLoading = true;

  // This function is used to fetch all data from the database
  void _refreshJournals() async {
    final List<Map<String, dynamic>> data = await SQLHelper.getItems();
    setState(() {
      _journals = data;
      _isLoading = false;
    });
  }

  @override
  void initState() {
    super.initState();
    _refreshJournals(); // Loading the diary when the app starts
  }

  final TextEditingController _titleController = TextEditingController();
  final TextEditingController _descriptionController = TextEditingController();

  // This function will be triggered when the floating button is pressed
  // It will also be triggered when you want to update an item
  //void _showForm(int? id) async { with null safty
  void _showForm(int? id) async {
    if (id != null) {
      // id == null -> create new item
      // id != null -> update an existing item
      final existingJournal =
        _journals.firstWhere((element) => element['id'] == id);
      _titleController.text = existingJournal['title'];
      _descriptionController.text = existingJournal['description'];
    }
  }

  showModalBottomSheet(
    context: context,
    elevation: 5,
    builder: (_) => Container(
      padding: const EdgeInsets.all(15),
      width: double.infinity,
      height: 300,
      child: SingleChildScrollView(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: [
            TextField(
              controller: _titleController,
              decoration: const InputDecoration(hintText: 'Title'),
            ),
            const SizedBox(
              height: 10,
            ),
            TextField(
              controller: _descriptionController,

```

```

        decoration:
          const InputDecoration(hintText: 'Description'),
      ),
      const SizedBox(
        height: 20,
      ),
      ElevatedButton(
        onPressed: () async {
          // Save new journal
          if (id == null) {
            await _addItem();
          }

          if (id != null) {
            await _updateItem(id);
          }

          // Clear the text fields
          _titleController.text = '';
          _descriptionController.text = '';

          // Close the bottom sheet
          Navigator.of(context).pop();
        },
        child: Text(id == null ? 'Create New' : 'Update'),
      ),
    ],
  ),
),
));
}

```

// Insert a new journal to the database

```

Future<void> _addItem() async {
  await SQLHelper.createItem(
    _titleController.text, _descriptionController.text);
  _refreshJournals();
}

```

// Update an existing journal

```

Future<void> _updateItem(int id) async {
  await SQLHelper.updateItem(
    id, _titleController.text, _descriptionController.text);
  _refreshJournals();
}

```

// Delete an item

```

void _deleteItem(int id) async {
  await SQLHelper.deleteItem(id);
  ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
    content: Text('Successfully deleted a journal!'),
  ));
  _refreshJournals();
}

```

@override

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Sqflite Roq App'),
    ),
  ),
}

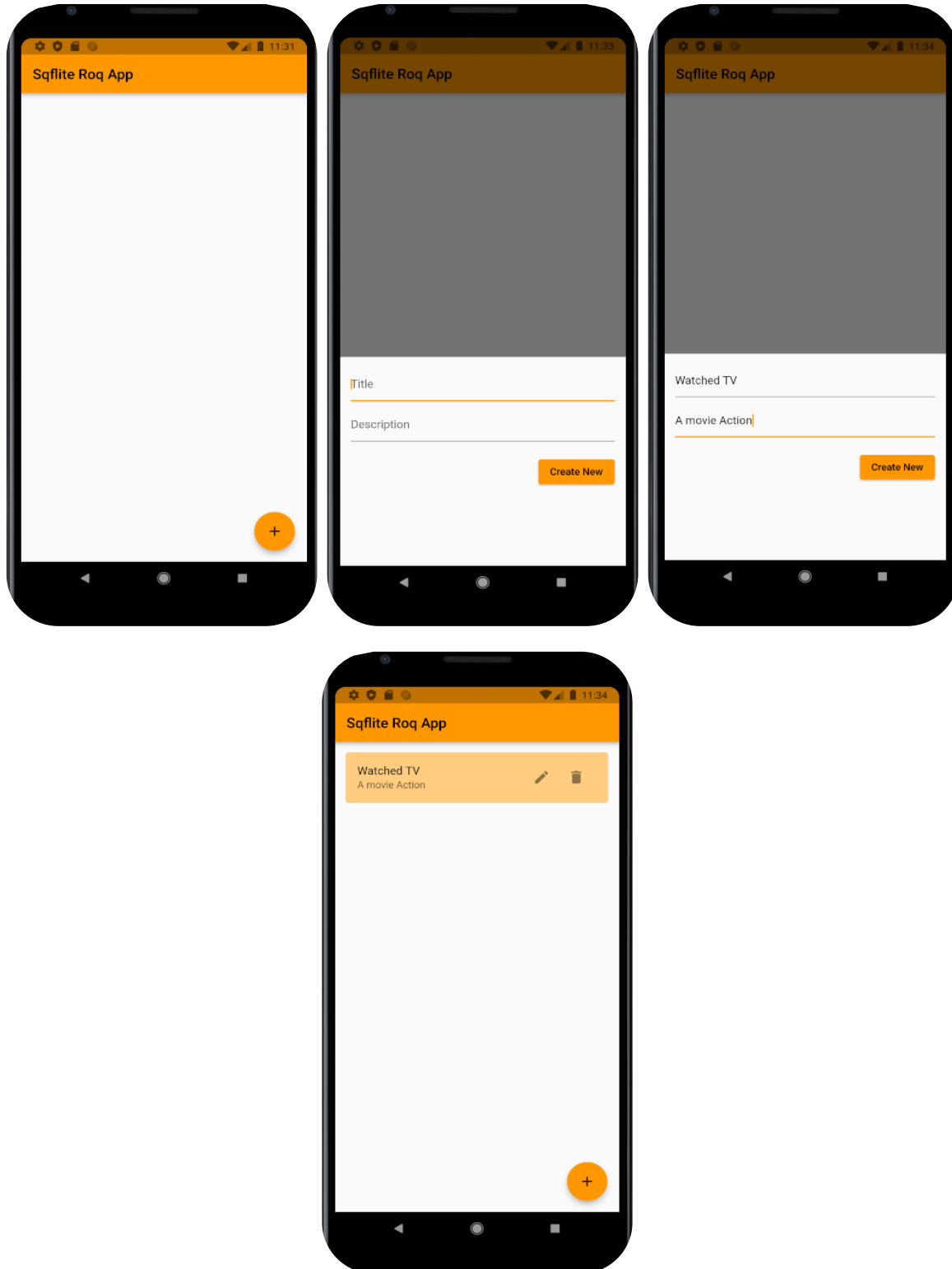
```

```

body: _isLoading
  ? const Center(
    child: CircularProgressIndicator(),
  )
  : ListView.builder(
    itemCount: _journals.length,
    itemBuilder: (context, index) => Card(
      color: Colors.orange[200],
      margin: const EdgeInsets.all(15),
      child: ListTile(
        title: Text(_journals[index]['title']),
        subtitle: Text(_journals[index]['description']),
        trailing: SizedBox(
          width: 100,
          child: Row(
            children: [
              IconButton(
                icon: const Icon(Icons.edit),
                onPressed: () => _showForm(_journals[index]['id']),
              ),
              IconButton(
                icon: const Icon(Icons.delete),
                onPressed: () =>
                  _deleteItem(_journals[index]['id']),
              ),
            ],
          ),
        ),
      ),
    ),
    floatingActionButton: FloatingActionButton(
      child: const Icon(Icons.add),
      onPressed: () => _showForm(null),
    ),
  );
}

```

Output



Persist data with SharedPreferences

SharedPreferences in flutter are used to store data locally in key-value pairs in android and ios apps. We might have to save data locally while developing applications. Data like login credentials or a flag etc. **SharedPreferences** comes in handy in such situations. For example, we have to save login credentials or a flag locally when the user logs in for the first time. This data will help the user access his account without logging in every time the app launches.



Flutter provides a plugin [shared_preferences](#) to implement sharedpreferences. It uses **NSUserDefaults** on ios and **sharedpreferences** in android. It provides persistent storage to save simple data. In this tutorial, we will learn how to implement and use sharedpreferences in flutter with example. We will also learn how to save, retrieve, and delete data locally using sharedpreferences.

Note: Don't use sharedpreferences to save complex data. It is useful to save simple data.

How To Implement SharedPreferences In Flutter ?

To implement and use sharedpreferences in flutter follow the steps below:

- Create a new flutter project.
- Add [shared_preferences](#) plugin or dependency to the project.
- Import the shared preferences package :- `import 'package:shared_preferences/shared_preferences.dart';`
- Implement the code as per your requirement like saving, retrieving, deleting, etc.

Let's add [shared_preferences](#) plugin or dependency to pubspec.yaml file present in the root folder of our project.

```
dependencies:  
  shared_preferences: <version>
```

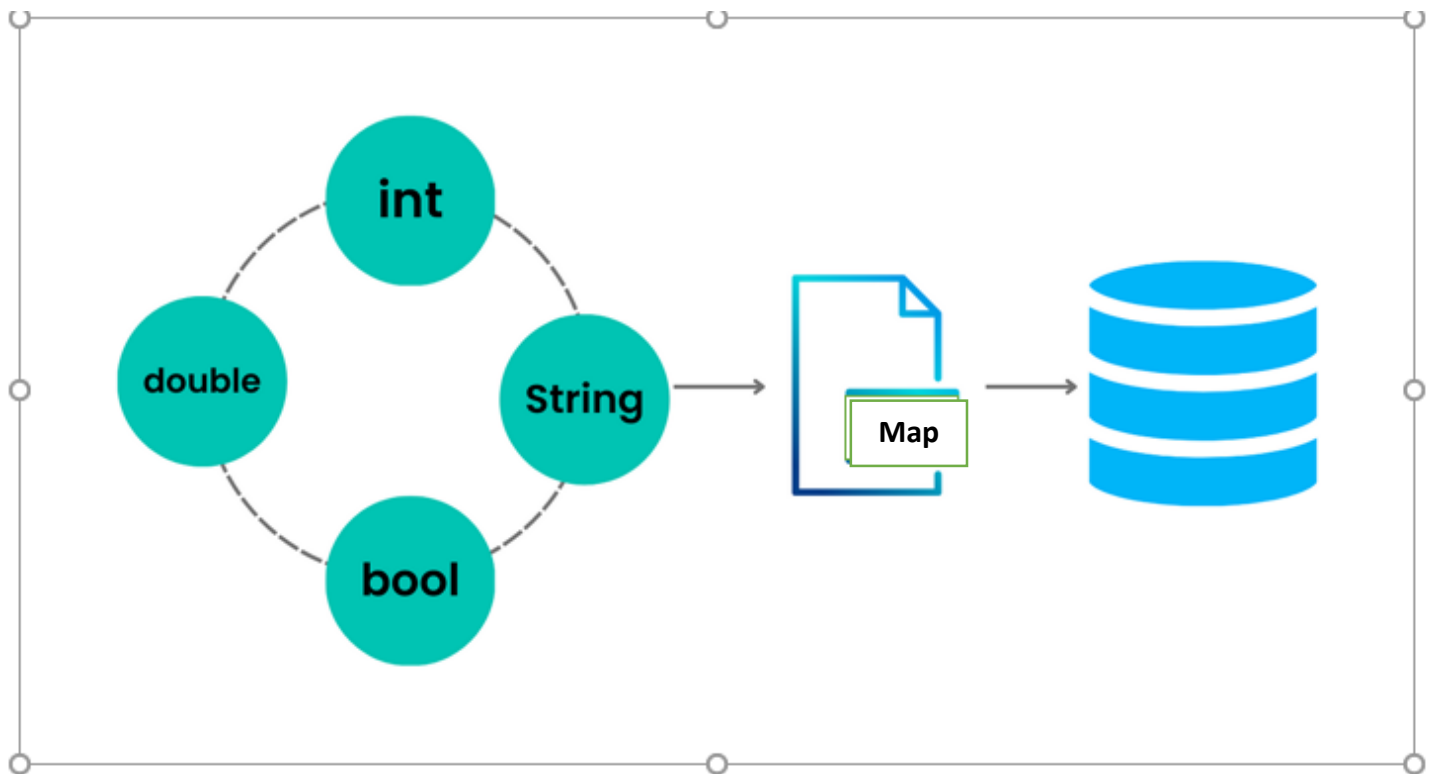
Import the package to use its functionality in our project.

```
import 'package:shared_preferences/shared_preferences.dart';
```

We are done adding the dependency and importing the package. Now we will see how to save, retrieve and delete data in sharedpreferences.

How To Save Data In SharedPreferences In flutter ?

To save data in sharedpreferences in flutter declare an object of SharedPreferences class. Initialize the object and call the setter methods to save data. Using sharedpreferences we can save five types of values **int**, **string**, **double**, **bool**, and **stringList**. So we have five setter methods for saving these five types of data. We have to provide each setter method with a key and a value. The key must be a string for all the methods. The values will change based on the method.



The setter methods of sharedPreferences class are:

- `setString()`
- `setInt()`
- `setDouble()`
- `setBool()`
- `setStringList()`

➤ **First, declare the object.**

```
late SharedPreferences prefs;
```

➤ **Second, Initialize the object.**

```
getSharedPreferences () async  
{
```

```
    prefs = await SharedPreferences.getInstance();  
}
```

Now we will use the above object to call the setter methods and save data in **SharedPreferences**.

Saving String value

To save **string** value in sharedpreferences call the **setString()** method and provide the key and value.

```
saveStringValue () async  
{  
    prefs = await SharedPreferences.getInstance();  
    prefs.setString("username", "Eng:Ruqaih Salman");  
}
```

The above code snippet will add **Eng:Ruqaih Salman** as a value to the key **username**.

Saving int value

To save **int** value in sharedpreferences we will use **setInt()** method. Call the method and provide the key and value.

```
saveIntValue () async  
{  
    prefs = await SharedPreferences.getInstance();  
    prefs.setInt("status", 1);  
}
```

The above code snippet will add **1** as a value to the key **status**.

Saving Double value

Call the setter method **setDouble()** and provide key and value to save a **double** value in sharedpreferences.

```
saveDoubleValue () async  
{  
    prefs = await SharedPreferences.getInstance();  
    prefs.setDouble("balance", 20.5);  
}
```

The above code snippet will add **20.5** as a value to the key **balance**.

Saving Boolean value

To save the boolean value in sharedpreferences call the **setBool()** method and provide the key and value.

```
saveBoolValue () async  
{  
    prefs = await SharedPreferences.getInstance();  
    prefs.setBool("loggedIn", true);  
}
```

The above code snippet will add **true** as a value to the key **loggedIn**.

Saving StringList value

To save a string list value in sharedpreferences call the **setStringList()** method and provide the key and value.

```
saveStringListValue () async
{
  prefs = await SharedPreferences.getInstance();
  prefs.setStringList("foodList", ["pizza", "burger", "sandwich"]);
}
```

The above code snippet will add list(["pizza", "burger", "sandwich"]) as a value to the key **foodList**.

How to Retrieve or Read Data from SharedPreferences?

To retrieve or read data from sharedpreferences in flutter we have to use getter methods of sharedpreferences class. There are five getter methods in sharedpreferences class. We have to provide the getter method with only the key which is a string. The method will return the value of that corresponding key.

The getter methods of sharedpreferences class are:

- getString()
- getInt()
- getDouble()
- getBool()
- getStringList()

While retrieving or reading data from shared preferences we might get null as a value. If the value is not present for the provided key or if we provide a wrong key it will return null. It will throw an error. We can solve this problem by using 'if null' (??). If the previous value is null it will return the value provided next to it. Let's see the implementation below.

```
prefs = await SharedPreferences.getInstance();
int users = prefs.getInt("count") ?? 0;
```

If `prefs.getInt("count")` returns a non null value it will be assigned to users variable. If it returns null as a value then '0' will be assigned to users variable. The `prefs.getInt("count")` will return null if no value is found for the key 'count' or if the key itself doesn't exist.

Retrieving or Reading String value

To retrieve or read **string** value from sharedpreferences call **getString()** method and provide the key.

```
retrieveStringValue () async
{
  prefs = await SharedPreferences.getInstance();
  String value = prefs.getString("username");
  print(value);
}
```

Output: Eng: Ruqaih Salman

Retrieving or Reading int value From

To retrieve or read **int** value from sharedpreferences call **getInt()** method and provide the key.

```
retrieveIntValue () async
{
  prefs = await SharedPreferences.getInstance();
  int value = prefs.getInt("status");
  print(value);
}
```

Output: 1

Retrieving or Reading Double value

To retrieve or read **double** value from sharedpreferences call **getDouble()** method and provide the key.

```
retrieveDoubleValue () async
{
  prefs = await SharedPreferences.getInstance();
  double value = prefs.getDouble("balance");
  print(value);
}
```

Output: 20.5

Retrieving or Reading Boolean value

To retrieve or read **boolean** value from sharedpreferences call **getBool()** method and provide the key.

```
retrieveBooleanValue () async
{
  prefs = await SharedPreferences.getInstance();
  bool value = prefs.getBool("loggedIn");
  print(value);
}
```

Output: true

Retrieving or Reading StringList value

To retrieve or read **string list** value from sharedpreferences call **getStringList()** method and provide the key.

```
retrieveStringListValue () async
{
  prefs = await SharedPreferences.getInstance();
  List<String> foodList = prefs.getStringList("foodList");
  print(value);
}
```

Output: ["pizza", "burger", "sandwich"]

How To Delete Data from SharedPreferences?

To delete or remove data from sharedpreferences in flutter we have to use **remove()** method. We have to provide only the key to delete the record (both the key and the value corresponding to that key). This method can delete all four types of data by simply taking the key.

```
deleteValue () async
{
  prefs = await SharedPreferences.getInstance();
  prefs.remove("username");
}
```

How to check if key is present or not in SharedPreferences ?

To check if a key or record is present in shared preferences we have to use **containsKey()** method. This method takes key as value and checks if the key is present in sharedpreferences. If the key is present it will return true else it will return false.

```
checkKey () async
{
  prefs = await SharedPreferences.getInstance();
  bool hasKey = prefs.containsKey("username");
  print(hasKey);
}
```

Output: true

```
checkKey () async
{
  prefs = await SharedPreferences.getInstance();
  bool hasKey = prefs.containsKey("counter");
  print(hasKey);
}
```

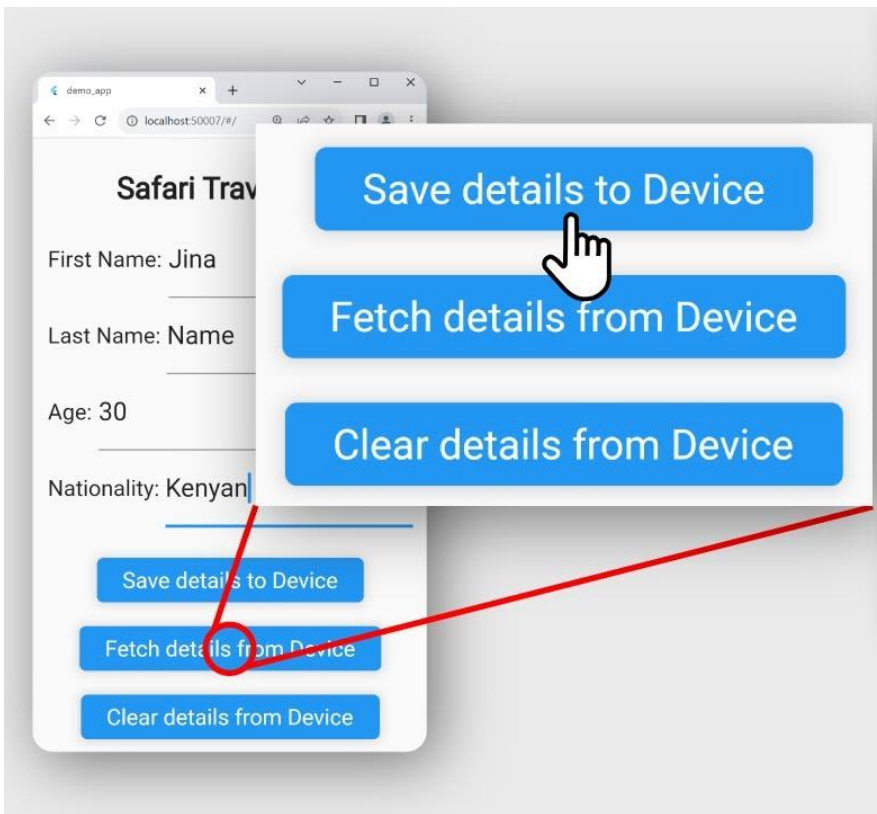
Output: false

➤ Example : SharedPreferences

Let's create an example where we will implement sharedpreferences in flutter. In this example we will display a textfield and a text widget. Below the text widget we will display three buttons save, retrieve and delete. We will save retrieve and delete the data entered in the textfield using sharedpreferences. When we press the save button we will save the data entered in the text field. When we press retrieve button we will retrieve the data from shared preferences and display it in the text widget. Delete button will clear the data in sharedpreferences.

SHARED PREFERENCES

IN FLUTTER



First, let's declare the variable for holding the text entered in the textfield and textediting controller. Also declare an object for SharedPreferences.

```
SharedPreferences prefs;  
TextEditingController controller = new TextEditingController();  
String name="";
```

Now implement the code for displaying the textfield, text widget and three buttons. Call the respective functions inside onPressed callback of each button.

```
body: Container(  
  margin: EdgeInsets.all(20),  
  child: Column(  
    children: [  
      TextField(  
        controller: controller,  
        decoration: InputDecoration(  
          border: OutlineInputBorder(),  
        ),  
      ),  
      SizedBox(  
        height: 30,  
      ),  
      Text(name, style: TextStyle(fontSize: 20)),  
      SizedBox(  
        height: 30,  
      ),  
      ElevatedButton(  
        child: Text("Save"),  
        onPressed: ()  
      )  
    ],  
  ),  
)
```

```
        save();
    },
    ElevatedButton(
        child:Text("retrieve"),
        onPressed: ()
        {
            retrieve();
        },
    ),
    ElevatedButton(
        child:Text("Delete"),
        onPressed: ()
        {
            delete();
        },
    )
),
),
```

- **Create three functions save, retrieve and delete.**

```
save() async
{
    prefs = await SharedPreferences.getInstance();
    prefs.setString("username", controller.text.toString());
}

retrieve() async{
    prefs = await SharedPreferences.getInstance();
    name = prefs.getString("username");
    setState(() {
        });
}

delete() async
{
    prefs = await SharedPreferences.getInstance();
    prefs.remove("username");
    name = "";
    setState(() {
        });
}
```

➤ **Example: Complete Code of sharedpreferences**

Lets sum up the above code snippets to complete our sharedpreferences example in flutter. Create a new flutter project and replace the code in main.dart file with the code below.

```
import 'package:shared_preferences/shared_preferences.dart';
import 'package:flutter/material.dart';
void main()
{
  runApp(Shared_Preferences_Roq_Ex2());
}

class Shared_Preferences_Roq_Ex2 extends StatelessWidget {
```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Ruqaih Salman',
    theme: ThemeData(
      primarySwatch: Colors.blue,
    ),
    home: MyHomePage(),
  );
}

class MyHomePage extends StatefulWidget {
  @override
  _MyHomePageState createState()
  {
    return _MyHomePageState();
  }
}

class _MyHomePageState extends State<MyHomePage> {
  late SharedPreferences prefs;
  TextEditingController controller = new TextEditingController();
  String name="";
  @override
  Widget build(BuildContext context) {
    return new Scaffold(
      appBar: AppBar(
        title: Text("SharedPreferences Roq App"),
      ),
      body: Container(
        margin: EdgeInsets.all(20),
        child: Column(
          children: [
            TextField(
              controller: controller,
              decoration: InputDecoration(
                border: OutlineInputBorder(),
              ),
            ),
            SizedBox(
              height: 30,
            ),
            Text(name!, style: TextStyle(fontSize: 20),),
            SizedBox(
              height: 30,
            ),
            ElevatedButton(
              child: Text("Save"),
              onPressed: ()
              {
                save();
              },
            ),
            ElevatedButton(
              child: Text("retrieve"),
              onPressed: ()
              {
                retrieve();
              },
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        ElevatedButton(
          child:Text("Delete"),
          onPressed: ()
            {
              delete();
            },
        ),
      ],
    ),
  ),
);
}

save() async
{
  /*setString(String key, String value)
  setInt(String key, int value):
  setDouble(String key, double value)
  setStringList(String key, List value)
  setBool(String key, bool value)*/
  prefs = await SharedPreferences.getInstance();
  prefs.setString("username", controller.text.toString());
}

retrieve() async{
  /*getString(String key)
  getInt(String key)
  getDouble(String key)
  getStringList(String key)
  getBool(String key)*/
  prefs = await SharedPreferences.getInstance();

  setState(() {
    name = prefs.getString("username")!;
  });
}

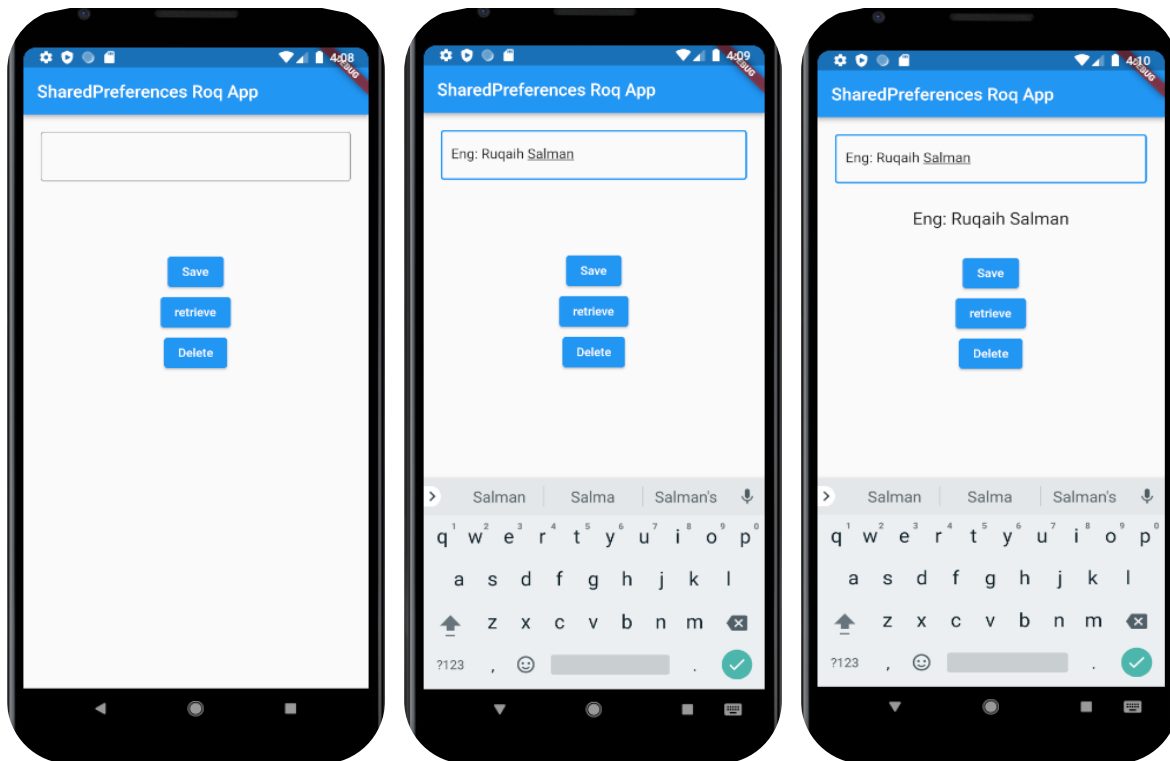
delete() async
{
  prefs = await SharedPreferences.getInstance();

  setState(() {
    prefs.remove("username");
    name = "";
  });
}
}

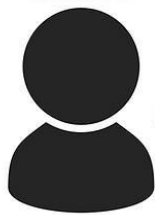
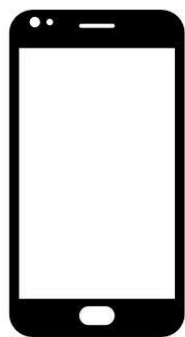
```

Since the data is saved locally in sharedpreferences it will not be deleted even if we kill the app. If we relaunch the app we can retrieve the data again.

Output:



we'll make a small Flutter application that lets the user enter his/her name and save it locally.



{
name
age
location



Shared
Preferences



➤ Example2:

When the app starts next time, it'll display a text like this:

Our app also has a button to reset the saved data (only shows up if has saved data).

1. Install the plugin by adding shared_preferences: ^0.5.12+4 to the dependencies section in your pubspec.yaml, like this:

```
dependencies:  
  flutter:  
    sdk: flutter  
  shared_preferences: ^0.5.12+4
```

Note that the indentation is mandatory. Then run this command:

```
flutter pub get
```

2. The complete code in main.dart:

```
import 'package:flutter/material.dart';  
import 'package:shared_preferences/shared_preferences.dart';  
  
void main() {  
  runApp(Shared_Preferences_Roq_Ex1());  
}  
  
class Shared_Preferences_Roq_Ex1 extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      // Hide the debug banner  
      debugShowCheckedModeBanner: false,  
      title: 'Ruqaih_Salman.com',  
      home: HomePage(),  
    );  
  }  
}  
  
class HomePage extends StatefulWidget {  
  @override  
  _HomePageState createState() => _HomePageState();  
}  
  
class _HomePageState extends State<HomePage> {  
  String savedName = '';  
  TextEditingController _nameController = TextEditingController();  
  
  // Retrieve the saved name if it exists  
  void initState() {  
    super.initState();  
    _retrieveName();  
  }  
  
  Future<void> _retrieveName() async {  
    final prefs = await SharedPreferences.getInstance();  
  
    // Check where the name is saved before or not  
    if (!prefs.containsKey('name')) {  
      return;  
    }  
  }  
}
```

```

    setState(() {
      savedName = prefs.getString('name')!;
    });
  }

Future<void> _saveName() async {
  final prefs = await SharedPreferences.getInstance();
  prefs.setString('name', _nameController.text);
}

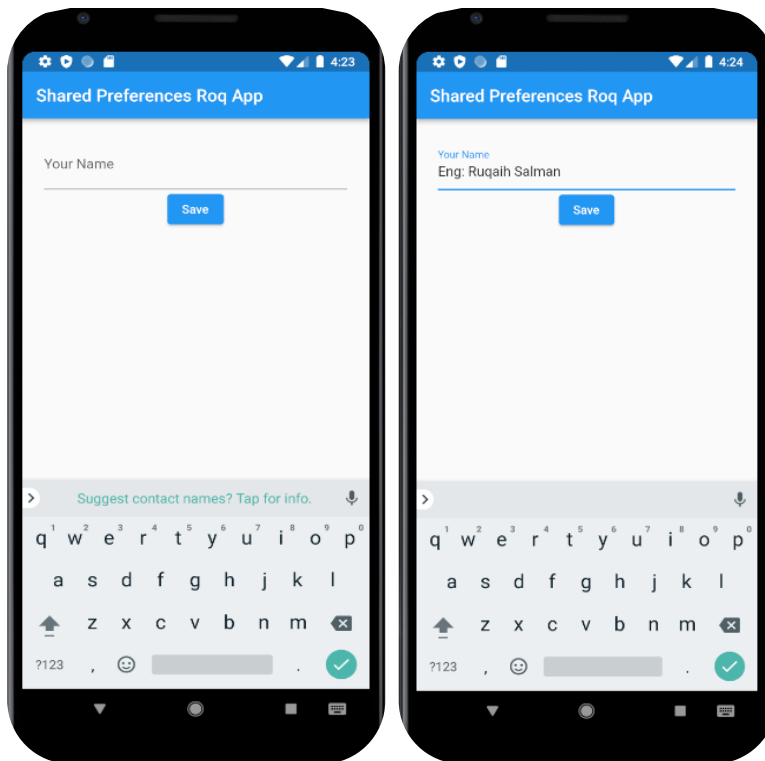
Future<void> _clearName() async {
  final prefs = await SharedPreferences.getInstance();
  // Check where the name is saved before or not
  if (!prefs.containsKey('name')) {
    return;
  }

  await prefs.remove('name');
  setState(() {
    savedName = '';
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Shared Preferences Roq App'),
    ),
    body: SafeArea(
      child: Padding(
        padding: const EdgeInsets.all(25),
        child: savedName == '' ? Column(
          children: [
            TextField(
              controller: _nameController,
              decoration: InputDecoration(labelText: 'Your Name'),
            ),
            ElevatedButton(
              onPressed: _saveName, child: Text('Save'))
          ],
        ) :
        Column(children: [
          Text(
            'Hello $savedName',
            style: TextStyle(fontSize: 50),
          ),
          ElevatedButton(
            onPressed: _clearName,
            child: Text('Reset'))
        ])),
    ));
}

```

Output:



Re-run the application, the results will appear as in the emulator below



Conclusion:

To store some simple data persistently, you can use the **shared_preferences** plugin which is officially published by the Flutter team. Data is persisted to disk asynchronously so you need to use **Future**, **async**, **await**.

Common methods of SharedPreferences

Writing data:

- `setString(String key, String value)`
- `setInt(String key, int value):`
- `setDouble(String key, double value)`
- `setStringList(String key, List value)`
- `setBool(String key, bool value)`

Retrieving data:

- `getString(String key)`
- `getInt(String key)`
- `getDouble(String key)`
- `getStringList(String key)`
- `getBool(String key)`
- `containsKey(String key):` Check whether **key** exists or not

Removing data:

- `remove(String key)`
- `clear():` Remove any saved data

Persist data with Text files (Read and write)

In some cases, you need to read and write files to disk. For example, you may need to persist data across app launches, or download data from the internet and save it for later offline use.

To save files to disk, combine the `path_provider` plugin with the `dart:io` library. This recipe uses the following steps:

- Find the correct local path.
- Create a reference to the file location.
- Write data to the file.
- Read data from the file.

1. Find the correct local path

This example displays a counter. When the counter changes, write data on disk so you can read it again when the app loads. Where should you store this data? The [path_provider](#) package provides a platform-agnostic way to access commonly used locations on the device's file system. The plugin currently supports access to two file system locations:

➤ Temporary directory

A temporary directory (cache) that the system can clear at any time. On iOS, this corresponds to the [NSCachesDirectory](#). On Android, this is the value that [getCacheDir\(\)](#) returns.

➤ Documents directory

A directory for the app to store files that only it can access. The system clears the directory only when the app is deleted. On iOS, this corresponds to the [NSDocumentDirectory](#). On Android, this is the [AppData](#) directory.

This example stores information in the documents directory. You can find the path to the documents directory as follows:

```
Future<String> _localPath() async {
  final directory = await getApplicationDocumentsDirectory();

  return directory.path;
}
```

2. Create a reference to the file location

Once you know where to store the file, create a reference to the file's full location. You can use the [File](#) class from the [dart:io](#) library to achieve this.

```
Future<File> _localFile() async {
  final path = await _localPath();
  return File('$path/counter.txt');
}
```

3. Write data to the file

Now that you have a [File](#) to work with, use it to read and write data. First, write some data to the file. The counter is an integer, but is written to the file as a string using the `'$counter'` syntax.

```
Future<File> writeCounter (int counter) async {
  final file = await _localFile();

  // Write the file
  return file.writeAsString('$counter');
}
```

4. Read data from the file

Now that you have some data on disk, you can read it. Once again, use the [File](#) class.

```
Future<int> readCounter() async {
  try {
```

```

    final file = await _localFile;

    // Read the file
    final contents = await file.readAsString();

    return (contents);
  } catch (e) {
    // If encountering an error, return 0
    return 0;
  }
}

```

➤ Example: Reading and Writing from file in Documents directory

Getting the file path

In this example, we don't need to manually create and add a text file to the project. It will be created automatically when we do writing data the first time. Here's the code in our **main.dart**:

The first step is: Install the package from the internet by adding **path_provider** and its version to the **dependencies** section of your **pubspec.yaml** file, like this:

```

dependencies:
  path_provider: ^1.6.27

```

Then run the following command:

```
flutter pub get
```

➤ Example: The full code and explanations of Files

```

// main.dart
import 'dart:convert';

import 'package:flutter/material.dart';
import 'dart:async';
import 'dart:io';
import 'package:path_provider/path_provider.dart';

void main() {
  runApp(Read_Write_from_file_Roq());
}

class Read_Write_from_file_Roq extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'File Roq App',
      home: HomePage(),
    );
  }
}

class HomePage extends StatefulWidget {
  @override
  _HomePageState createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {

```

```

// This will be displayed on the screen
String? _content;

// Find the Documents path
Future<String> _getDirPath_funroq() async {
  final _dir = await getApplicationDocumentsDirectory();
  print("The path of file is : ${_dir.path}");
  return _dir.path;
}

// This function is triggered when the "Read" button is pressed
Future<void> _readData_funroq() async {
  final _dirPath_roq = await _getDirPath_funroq();
  final _myFile_roq = File('${_dirPath_roq}/data.txt');
  final _data_roq = await _myFile_roq.readAsString(encoding: utf8);
  setState(() {
    _content = _data_roq;
  });
}

// TextField controller
final _textController = TextEditingController();

// This function is triggered when the "Write" button is pressed
Future<void> _writeData_funroq() async {
  final _dirPath_roq = await _getDirPath_funroq();

  final _myFile_roq = File('${_dirPath_roq}/data.txt');
  // If data.txt doesn't exist, it will be created automatically

  await _myFile_roq.writeAsString(_textController.text);
  _textController.clear();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Textfile Roq App'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: [
          TextField(
            controller: _textController,
            decoration: InputDecoration(labelText: 'Enter your name'),
          ),
          ElevatedButton(
            child: Text('Save to file'),
            onPressed: _writeData_funroq,
          ),
          SizedBox(
            height: 150,
          ),
          Text(
            _content != null
              ? _content!
              : 'Press the button to load your name',
            style: TextStyle(fontSize: 24, color: Colors.pink),
          ),
          ElevatedButton(

```

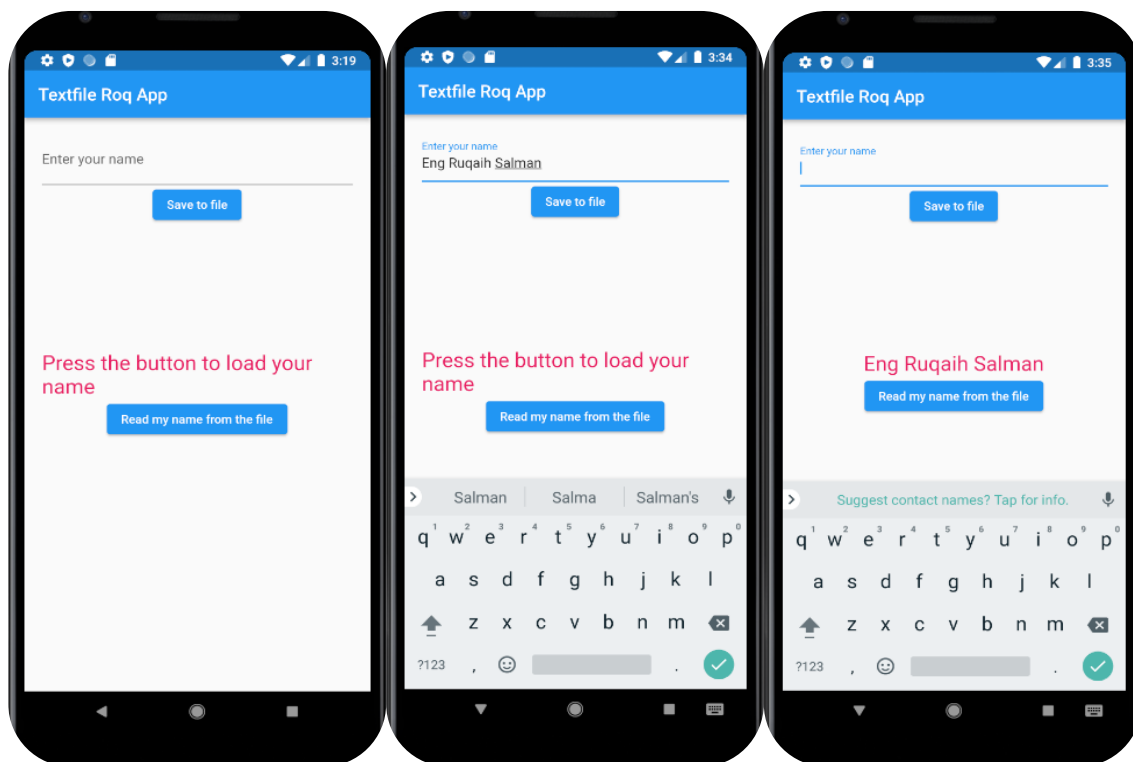
```

        child: Text('Read my name from the file'),
        onPressed: _readData_funroq,
      ),
    ),
  ),
);
}
}

```

Output:

This example app has a TextFiled that lets the user enter his/her name to write to a text file when pressing the **Save** button. It also contains a Text widget that displays the name read from that file when pressing the **Read** button.



Read data from local files

Files (which have **.txt, .csv or .json** extension) are widely used to store information persistently, from numerical data to long text. In lab9, we will walk through 3 examples of Flutter applications which work with these file types. The examples are quick and simple. It only loads content from a text, csv or json file that is found in the assets folder (or another folder in the root project) using **rootBundle** (from **services.dart**) then outputs the result onto the screen. This is useful when you only need to read data and not write data.

➤ **Q: How to read data from local (Text, Csv, or JSON) file in Flutter? =>(Run Lab9)**